



信息科学与技术学院

School of Information Science and Technology

CS 110

Computer Architecture Operating System & I/O

Instructors:

Chundong Wang, Siting Liu & Yuan Xiao

Course website: <https://toast->

[lab.sist.shanghaitech.edu.cn/courses/CS110@ShanghaiTech/Spring-2025/index.html](https://toast-lab.sist.shanghaitech.edu.cn/courses/CS110@ShanghaiTech/Spring-2025/index.html)

School of Information Science and Technology (SIST)

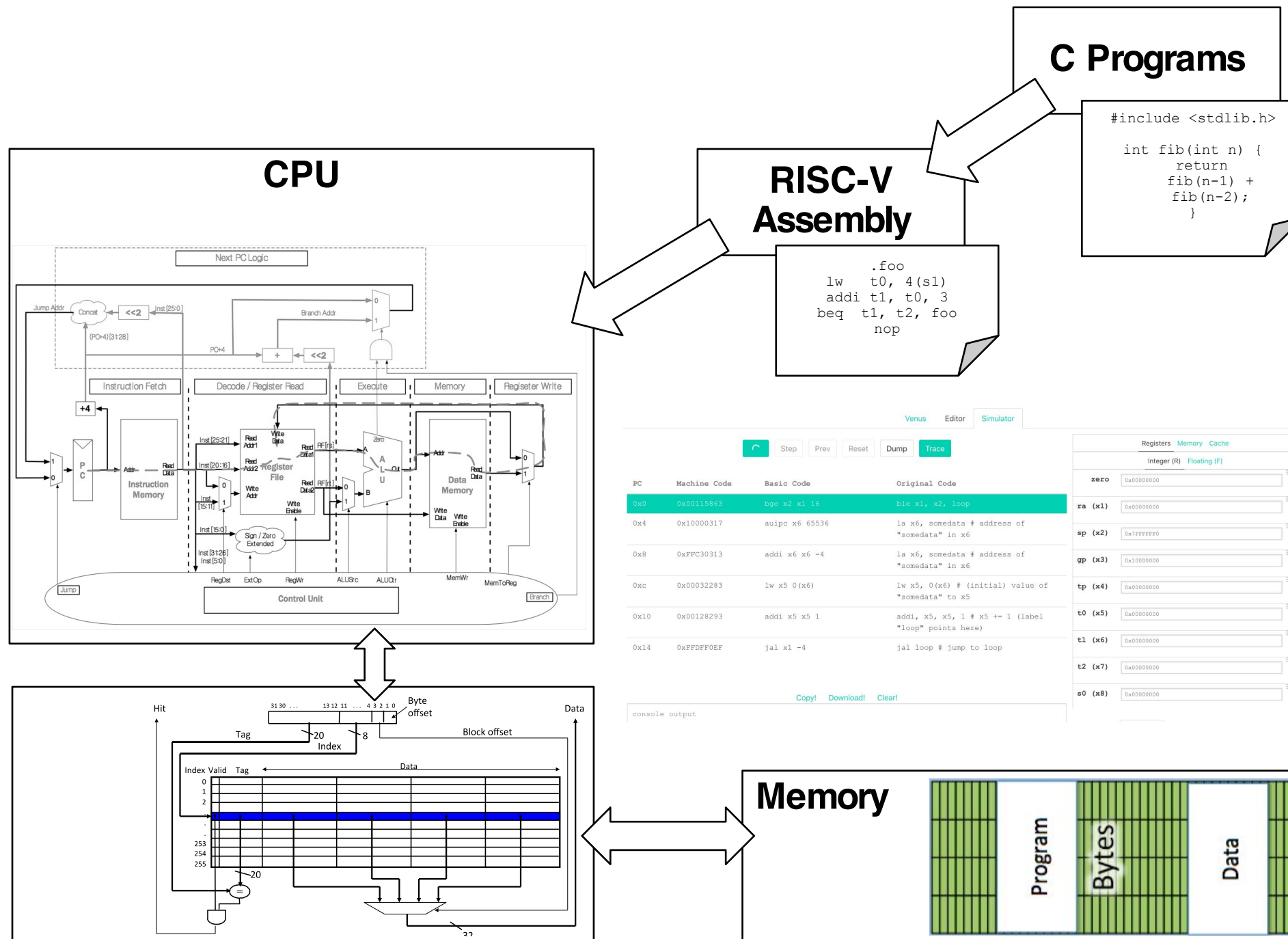
ShanghaiTech University

2025/5/22

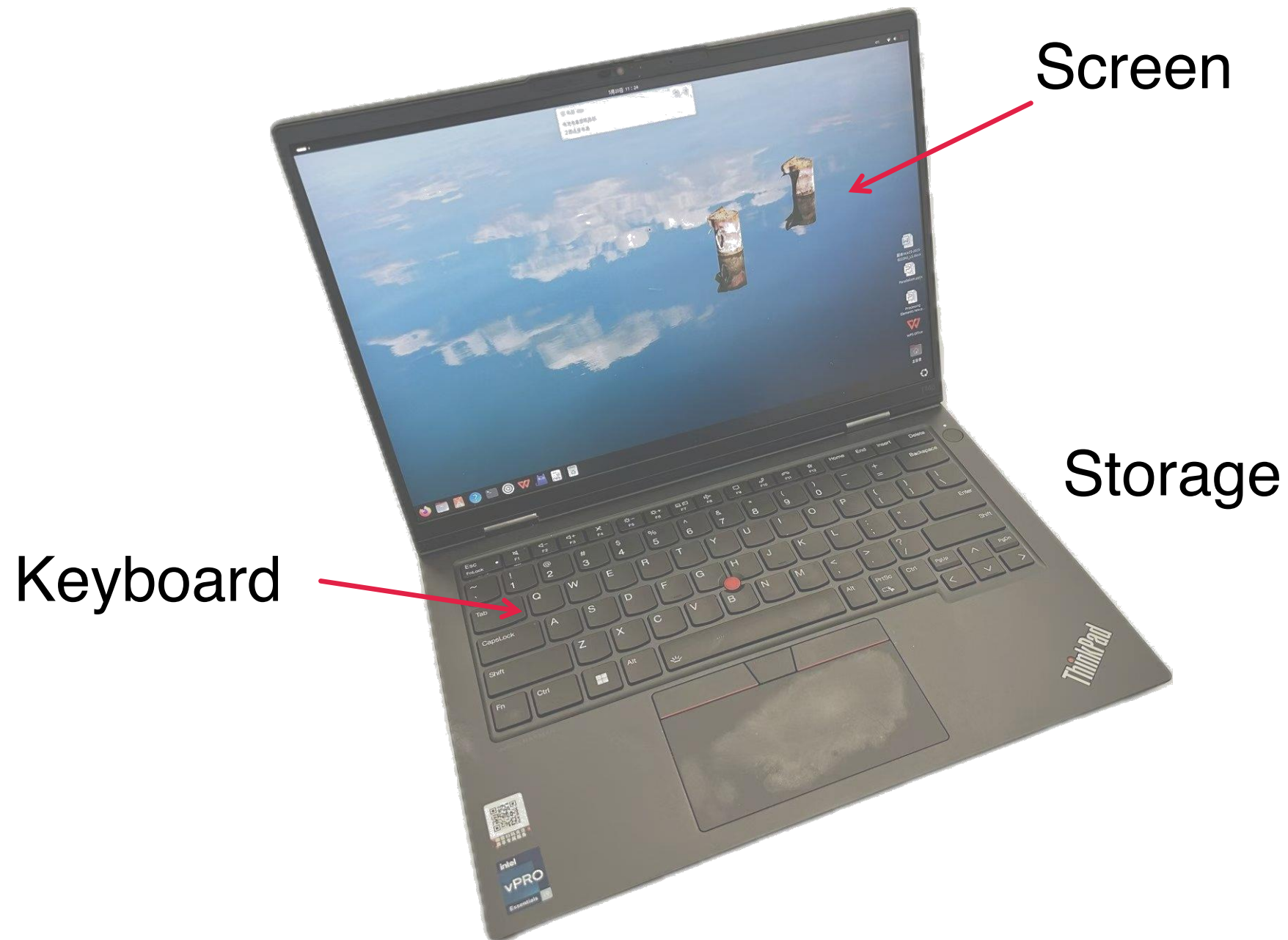
Administratives

- Final exam, June 12th 8am-10am; you can bring **3**-page A4-sized double-sided cheat sheet, **handwritten** only! (**Teaching center 201/202/203**); the whole course will be covered. No electronic devices (no smart watches, no calculators, etc.)
- Project 3 released. Speed Competition! ddl May 29th.
- Project 4 released, ddl June 3rd.
- HW 7 released, ddl approaching, May 30th.
- HW 8 will be released soon, ddl June 5th.
- To check Lab 13 this week, May 20th, 22nd & 26th
- Lab 14 released, to check May 27th, 29nd & June 4th (Lab Session 1 only, 1D104); **Prepare in advance!**
- Discussion May 23th & 26th on *OS & Virtual memory*.

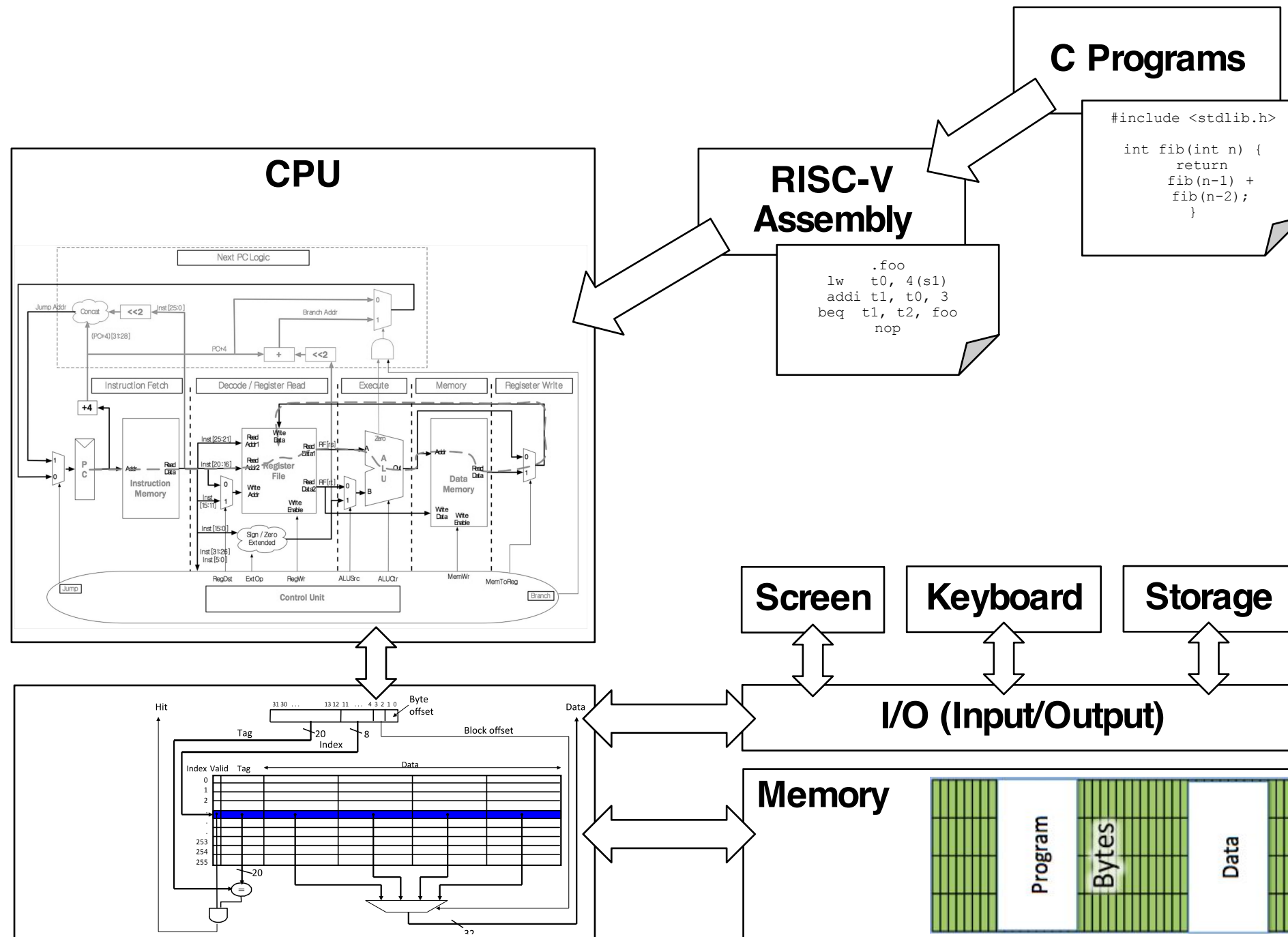
CA so far ...



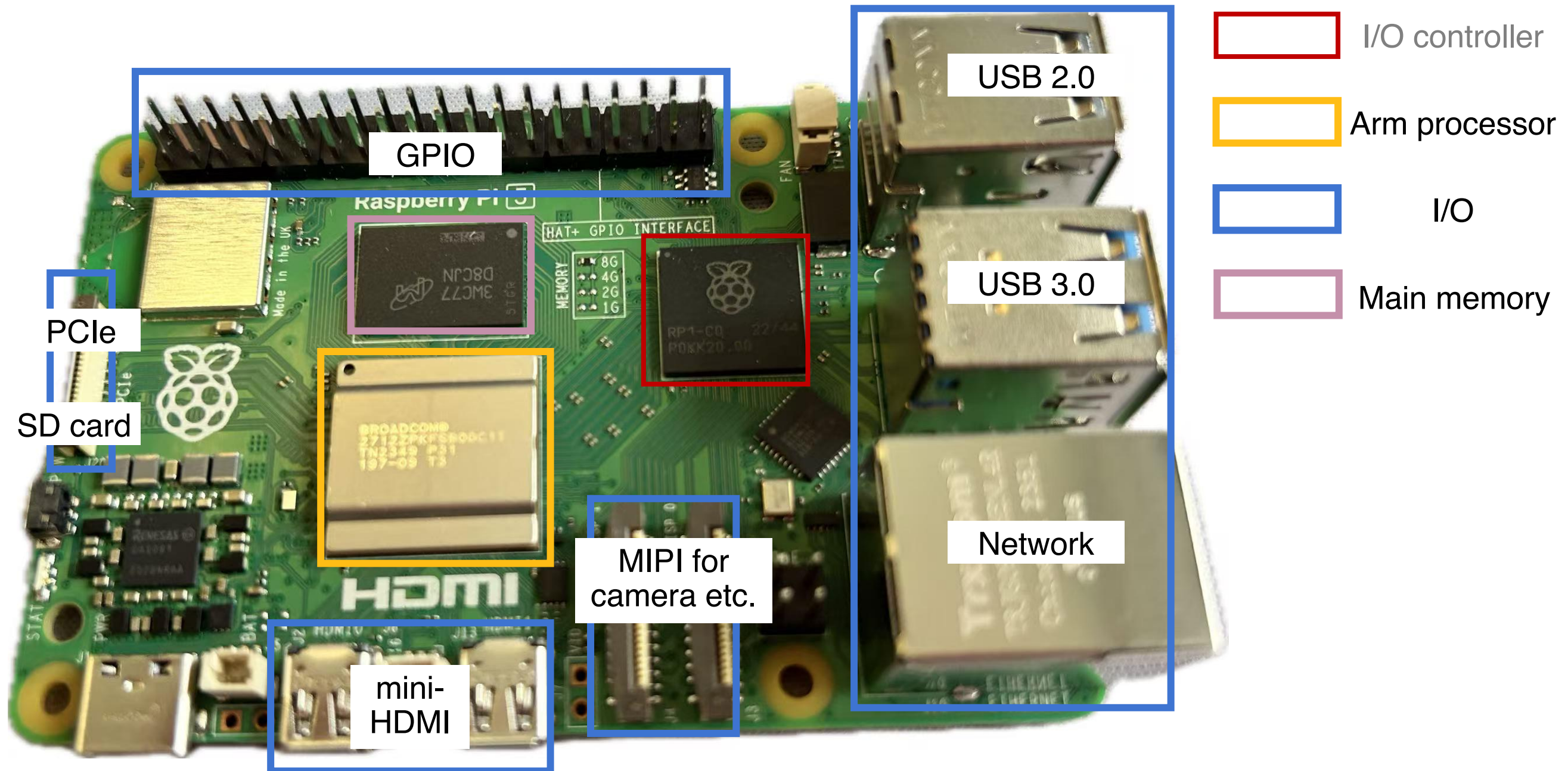
Different from Real Staff



Add I/O



Raspberry Pi



It is a real computer!



Just Software?

- The biggest piece of software on your machine?
- How many lines of code? These are guesstimates (in millions of lines of code)



CC BY-NC 3.0
 David McCandless©2015
<http://www.informationisbeautiful.net/visualizations/million-lines-of-code/>

Linux Kernel Over Time

- OS $\approx$ kernel
 - User interface and other components not covered

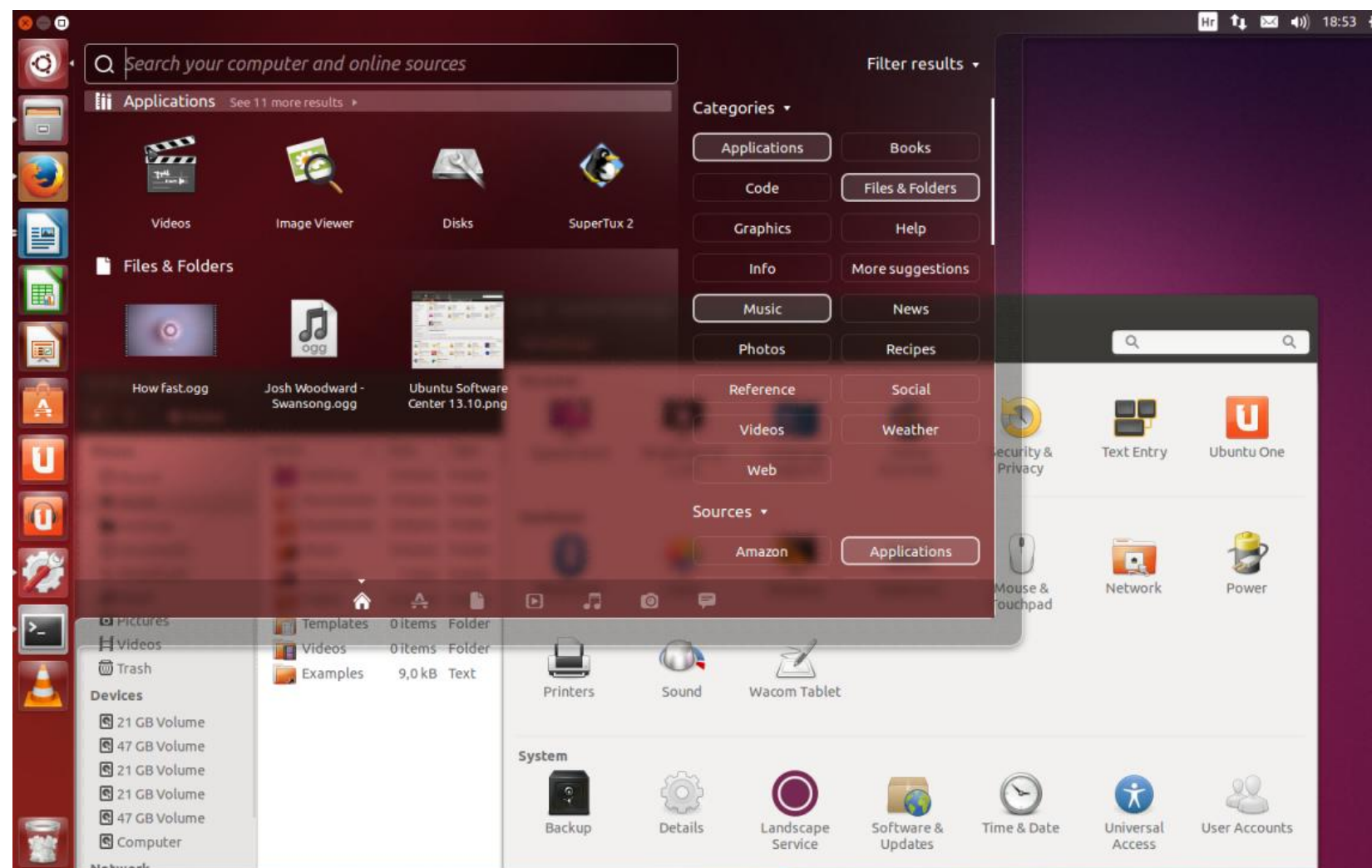
Year	Kernel Version	Size of zipped file
1994	linux-1.0.tar.gz	1MB
1996	linux-2.0.tar.gz	6MB
2001	linux-2.4.0.tar.gz	23MB
2003	linux-2.6.0.tar.gz	40MB
2011	linux-3.0.tar.gz	92MB
2015	linux-4.0.tar.gz	118MB
2019	linux-5.0.tar.gz	155MB
Apr 2020	linux-5.6.8.tar.gz	166MB
May 2022	linux-5.17.5.tar.gz	189MB
May 2024	Linux-6.9.tar.gz	222MB
May 2025	Linux-6.14.7.tar.gz	231MB

All 7 fictions in txt format
zipped to be **2.5MB**



Besides the Hardware

- That's still not the same. CS 110 experience isn't like the real world.
- When switching on a computer, get this



Yes, that is just the software!
The operating system (OS)

What does the OS do?

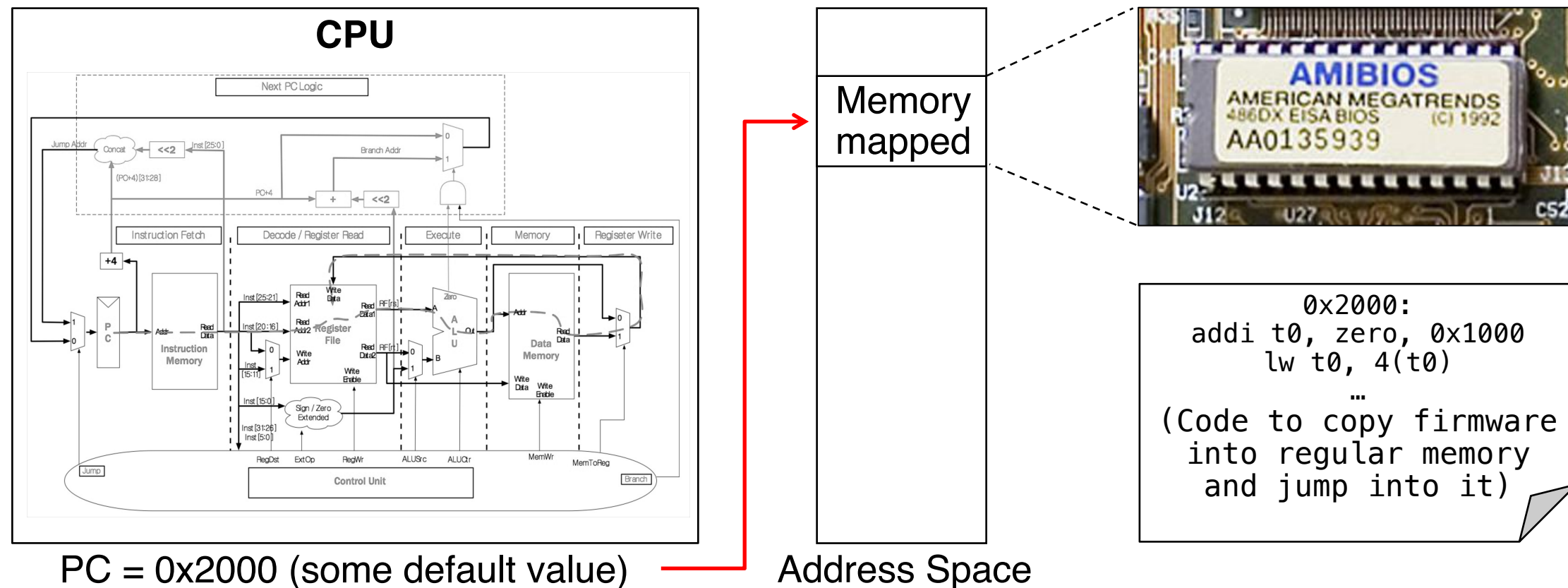
- One of the first things that runs when your computer starts (right after firmware/bootloader)
 - Provides services (100+): File System, Network stack, printer, etc.
- Loads, runs and manages programs (recall CALL):
 - Isolate programs from each other (isolation), each program/process runs (appears to run) in its own little world; (later, virtual memory)
 - Multiplex resources between applications (e.g., hardware threads memory, I/O devices: disk, keyboard, display, network, etc.)
- I/O with the rest of computer
 - Provide interaction with the outside world;
 - Finds and controls all the devices in the machine in a general way (using “device drivers”)

What does the OS do?

- One of the first things that runs when your computer starts (right after firmware/bootloader)
- Loads, runs and manages programs
- I/O with the rest of computer

What happens at Boot?

- When the computer switches on, the CPU executes instructions from some start address (stored in Flash ROM)



- Bootstrapping: <https://en.wikipedia.org/wiki/Bootstrapping>

What happens at Boot?

- When the computer switches on, the CPU executes instructions from some start address (stored in Flash ROM)

1. BIOS: Find a storage device and load first sector (block of data)

```

Diskette Drive B : None          Serial Port(s) : 3F0 2F0
Pri. Master Disk : LBA,ATA 100, 250GB Parallel Port(s) : 370
Pri. Slave Disk : LBA,ATA 100, 250GB DDR at Bank(s) : 0 1 2
Sec. Master Disk : None
Sec. Slave Disk : None

Pri. Master Disk HDD S.M.A.R.T. capability ... Disabled
Pri. Slave Disk HDD S.M.A.R.T. capability ... Disabled

PCI Devices Listing ...
Bus Dev Fun Vendor Device SVID SSID Class Device Class IRQ
0 27 0 8086 2668 1458 A005 0403 Multimedia Device 5
0 29 0 8086 2658 1458 2658 0C03 USB 1.1 Host Cntrlr 9
0 29 1 8086 2659 1458 2659 0C03 USB 1.1 Host Cntrlr 11
0 29 2 8086 265A 1458 265A 0C03 USB 1.1 Host Cntrlr 11
0 29 3 8086 265B 1458 265A 0C03 USB 1.1 Host Cntrlr 5
0 29 7 8086 265C 1458 5006 0C03 USB 1.1 Host Cntrlr 9
0 31 2 8086 2651 1458 2651 0101 IDE Cntrlr 14
0 31 3 8086 266A 1458 266A 0C05 SMBus Cntrlr 11
1 0 0 10DE 0421 10DE 0479 0300 Display Cntrlr 5
2 0 0 1283 8212 0000 0000 0180 Mass Storage Cntrlr
2 5 0 11AB 4320 1458 E000 0200 Network Cntrlr
ACPI Controller
  
```

2. Bootloader (stored on, e.g., disk): Load the OS *kernel* from disk into a location in memory and jump into it.

```

QUESTION 3:
conv: <speedup> x
relu: <speedup> x
pool: <speedup> x
fc: <speedup> x
softmax: <speedup> x

Which layer should we optimize?
<which layer>

(23:04:03 Wed Apr 15 2015 cs61c-ti@hive22 Linux x86_64)
~/src/proj3/proj3_starter $ ls src/
answers.txt cnn cnnModule.py data LICENSE Makefile just web

(23:04:09 Wed Apr 15 2015 cs61c-ti@hive22 Linux x86_64)
~/src/proj3/proj3_starter $ ls src/
cnn.c main.c python.c util.c

(23:04:16 Wed Apr 15 2015 cs61c-ti@hive22 Linux x86_64)
~/src/proj3/proj3_starter $ make cnn
make: 'cnn' is up to date.

(23:04:20 Wed Apr 15 2015 cs61c-ti@hive22 Linux x86_64)
~/src/proj3/proj3_starter $
  
```

4. Init: Launch an application that waits for input in loop (e.g., Terminal/Desktop/...

```

the KNOPPIX live GNU/Linux on DVD!

Running Linux Kernel 2.6.24.4.
Total Memory available: 124132kB, Memory free: 118189kB.
Scanning for USB/Firewire devices... Done.
Enabling DMA acceleration for: hdc [QEMU CD-ROM]
Accessing KNOPPIX DVD at /dev/hdc...
Found primary KNOPPIX compressed image at /cdrom/KNOPPIX/KNOPPIX.
Found additional KNOPPIX compressed image at /cdrom/KNOPPIX/KNOPPIX2.
Creating /ramdisk (dynamic size=99304k) on shared memory...Done.
Creating unified filesystem and symlinks on ramdisk...
>> Read-only DVD system successfully merged with read-write /ramdisk.
Done.
Starting INIT (process 1).
INIT: version 2.86 booting
Configuring for Linux Kernel 2.6.24.4.
Processor 0 is Pentium II (Klamath) 1662MHz, 128 KB Cache
apm (1600): apm 3.2.1 interfacing with apm driver 1.16ac and APM BIOS 1.2
Acs found, power management functions enabled.
Acs found, managed by udev
Acs found, managed by udev
udev hot-plug hardware detection... Started.
Configuring devices...
  
```

3. OS Boot: Initialize services, drivers, etc.

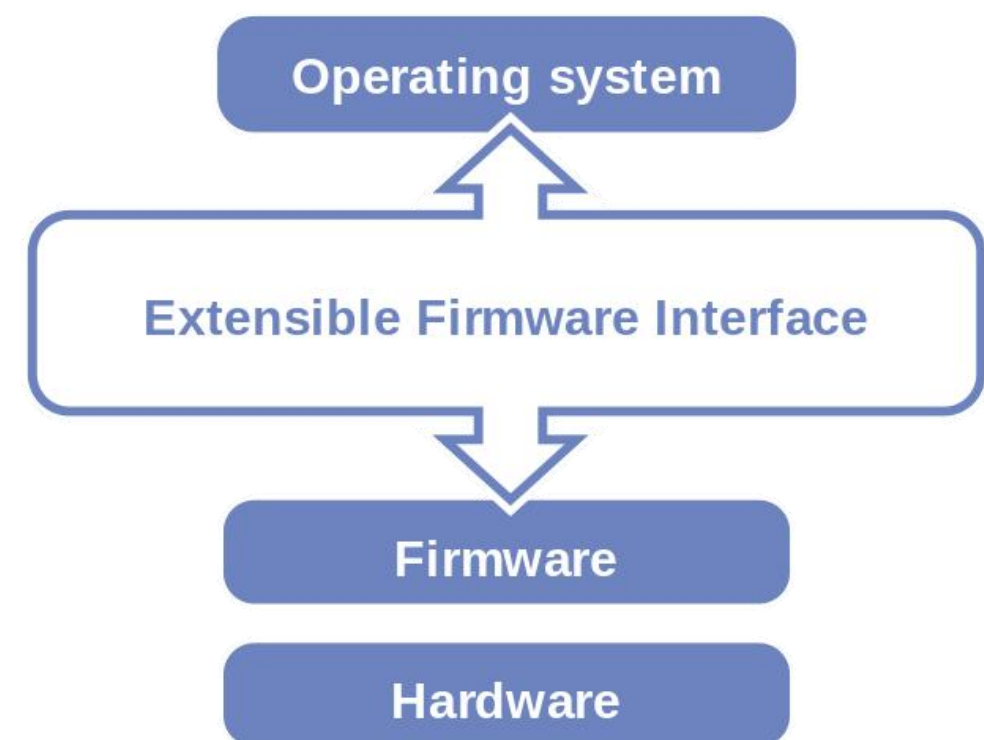
```

.04, kernel 2.6.24-16-generic
.04, kernel 2.6.24-16-generic (recovery mode)
.04, memtest86+

Use the ↑ and ↓ keys to select which entry is highlighted.
Press enter to boot the selected OS, 'e' to edit the
commands before booting, or 'c' for a command-line.
  
```

UEFI: Unified Extensible Firmware Interface

- Successor of BIOS
- Much more powerful and complex
- E.g. graphics menu; networking; browsers
- All modern Intel & AMD-based computer use UEFI

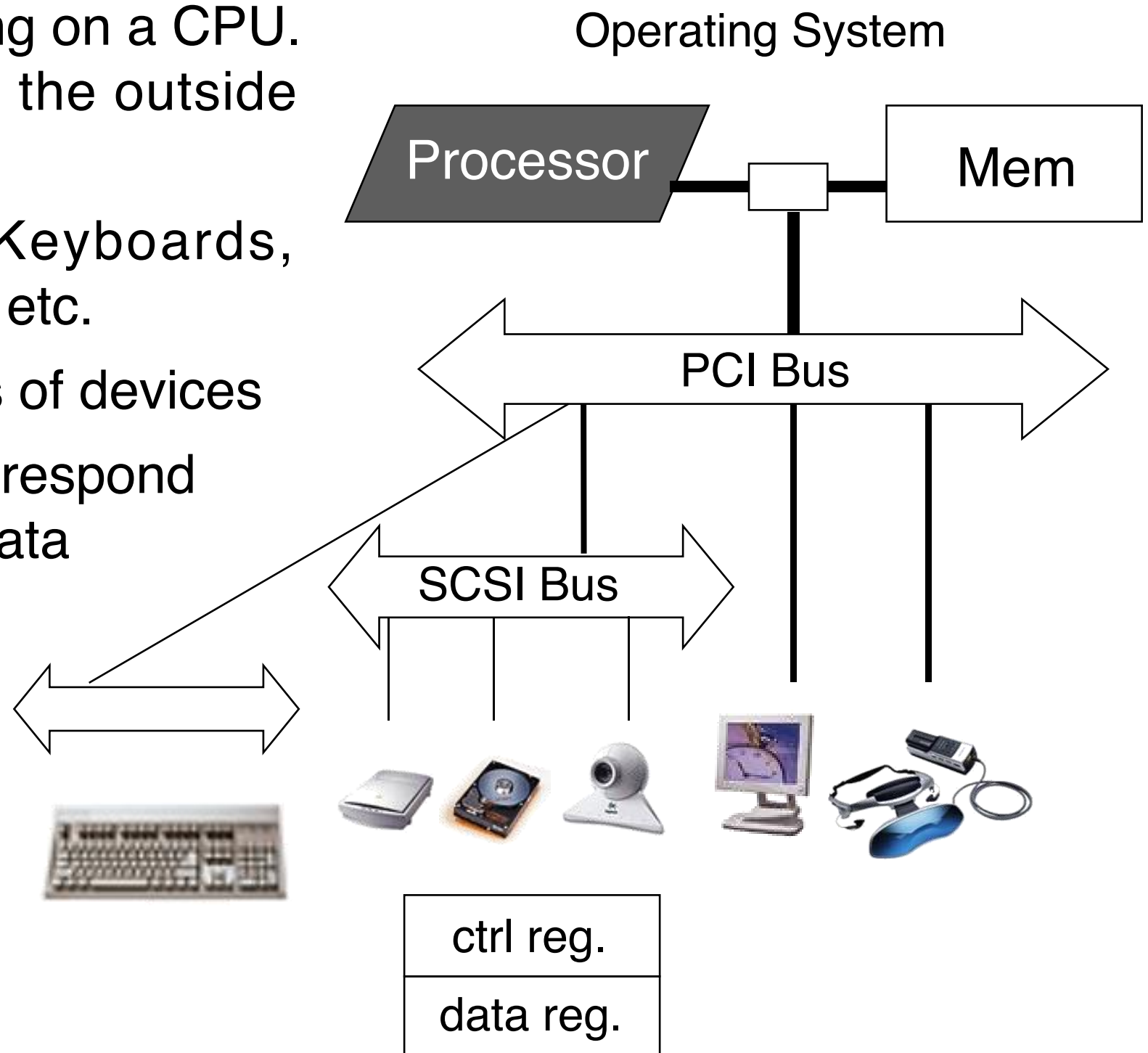


What does the OS do?

- One of the first things that runs when your computer starts (right after firmware/bootloader)
- Loads, runs and manages programs
- I/O with the rest of computer

How to interact with devices?

- Assume a program running on a CPU. How does it interact with the outside world?
- Need I/O interface for Keyboards, Network, Mouse, Screen, etc.
 - Connect to many types of devices
 - Control these devices, respond to them, and transfer data
 - Present them to user programs so they are useful

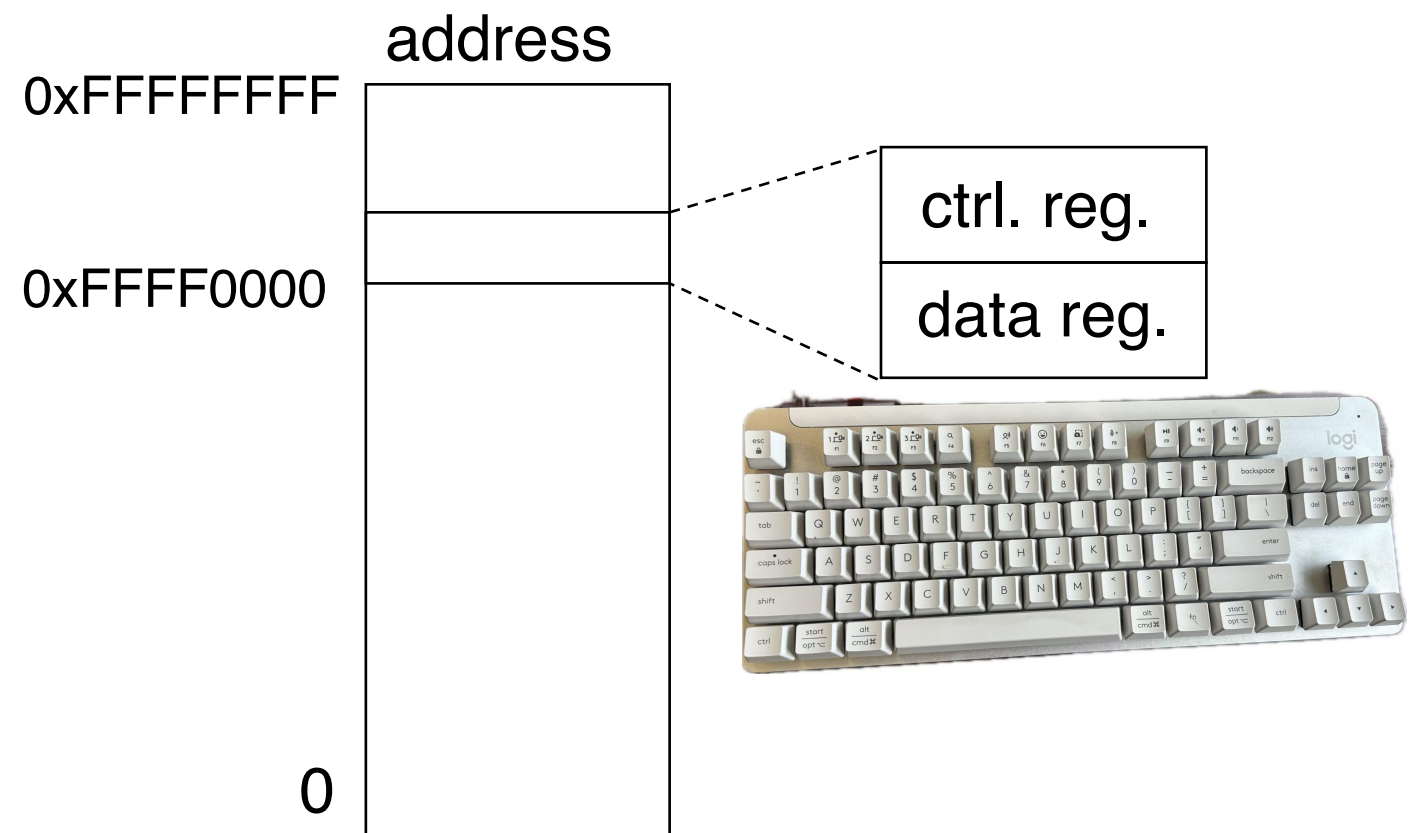


Instruction Set Architecture for I/O

- Could define a separate scheme to handle each different I/O device ...
 - ...or we could **standardize the interface** and let the I/O device handle any complications.
- What must the processor do for I/O?
 - Input: reads a sequence of bytes
 - Output: writes a sequence of bytes
- Interface options
 - Some processors have special input/output instructions
 - **Memory Mapped Input/Output** (MMIO, used by RISC-V):
 - Use normal load/store instructions, e.g., `lw/sw`, for input/output
 - In small pieces
 - A portion of the address space dedicated to I/O
 - I/O device registers there (no memory there)

Memory Mapped I/O

- Certain addresses are not regular memory, instead, they correspond to registers in I/O devices
- Shared abstraction: I/O devices read/write bytestreams.
- Shared interface is therefore memory mapped I/O
- This is a portion of address space dedicated to I/O that does not contain “regular” memory; rather, it corresponds to registers in I/O devices!
- Makes it easy to use normal load/store instructions, e.g. `lw/sw`
 - (Very common, used by RISC-V)



Caveat: Speed Mismatch

- In theory, simple. In practice, extremely difficult to standardize over 9 orders of magnitude of data rate!
- 1 GHz microprocessor I/O throughput: 4 GiB/s (hw/sw)
- Some I/O peak data rates:
 - 10 B/s (keyboard)
 - 3 MiB/s (Bluetooth 3.0)
 - 0.06-1.25 GiB/s (USB 2/3.1)
 - 7-250 MiB/s (WIFI, depends on standard)
 - 125 MiB/s (G-bit Ethernet)
 - 480 MiB/s (SATA3 HDD)
 - 560 MiB/s (cutting edge SSD)
 - 5 GiB/s (Thunderbolt 3)
 - 32 GiB/s (High-end DDR4 DRAM)
 - 64 GiB/s (HBM2 DRAM)
- Input may be waiting for human to act
- Output device may not be ready to accept data (as fast as processor stores it)

Processor: Polls vs. Interrupts

- Polling
 - e.g., “30 times per second”
- Processor reads from control register in loop
 - Wait for device to set ready bit in control reg. (0→1) indicates “data available” (for input device) or “ready to accept data” (for output device);
 - Then loads from/writes to data reg.
 - I/O device resets control reg. (1→0)
- Interrupts
 - Avoid wasting processor resources for low data rate devices (e.g., mouse, keyboard)
- Processor runs as usual
 - Occurs when I/O is ready
 - Interrupt current program
 - Transfer control to the trap handler in the operating system

I/O Polling Example

- Input: Read from keyboard into **a0**

```
        li    t0, 0xffff0000 #ffff0000
Waitloop: lw    t1, 0(t0)      #control
        andi  t1, t1, 0x1
        beq   t1, zero, Waitloop
        lw    a0, 4(t0)      #data
```

- Output: Write to display from **a0**

```
        li    t0, 0xffff0000 #ffff0000
Waitloop: lw    t1, 8(t0)     #control
        andi  t1, t1, 0x1
        beq   t1, zero, Waitloop
        sw    a0, 12(t0)     #data
```

“Ready” bit is from processor’s point of view!

Cost of Polling

- Assume for a processor with a 1GHz clock it takes 400 clock cycles for a polling operation (call polling routine, accessing the device, and returning). Determine % of processor time for polling
 - Mouse: polled 30 times/sec so as not to miss user movement

Cost of Polling

- Assume for a processor with a 1GHz clock it takes 400 clock cycles for a polling operation (call polling routine, accessing the device, and returning). Determine % of processor time for polling
 - Mouse: polled 30 times/sec so as not to miss user movement
- Mouse Polling [clocks/sec]
 $= 30 \text{ [polls/s]} * 400 \text{ [clocks/poll]} = 12\text{K [clocks/s]}$
- % Processor for polling:
 $12 * 10^3 \text{ [clocks/s]} / 1 * 10^9 \text{ [clocks/s]} = 0.0012\%$
 $\Rightarrow$ Polling mouse **little** impact on processor

Alternative to Polling: Interrupts

- Wasteful to have processor spend most of its time “spin-waiting” for I/O to be ready
- Would like an unplanned procedure call that would be invoked only when I/O device is ready
- Solution: use **interrupts** to help I/O.
 - **Interrupt** program when I/O ready, return when done with data transfer
- Allow to register (post) **interrupt handlers**: functions that are called when an interrupt is triggered

Interrupt-driven I/O

Low

Handler Execution

Stack Frame

Stack Frame

Stack Frame

High

1. Incoming interrupt suspends instruction stream
2. Looks up the vector (function address) of a handler in **an interrupt vector table** stored within the CPU
3. Perform a `jal` to the handler (**needs to store any state**)
4. Handler run on current stack and returns on finish (thread doesn't notice that a handler was run)

```

Label: sll  t1,s3,2
      addu t1,t1,s5
      lw   t1,0(t1)
      add  s1,s1,t1
      addu s3,s3,s4
      bne  s3,s2,label
  
```

```

handler:  li t0, 0xffff0000
          lw t1, 0(t0)
          andi t1, t1, 0x1
          lw a0, 4(t0)
          sw t1, 8(t0)
          ret
  
```

Interrupt
(SPI0)

CPU Interrupt Table	
SPI0	handler
...	...

Terminologies

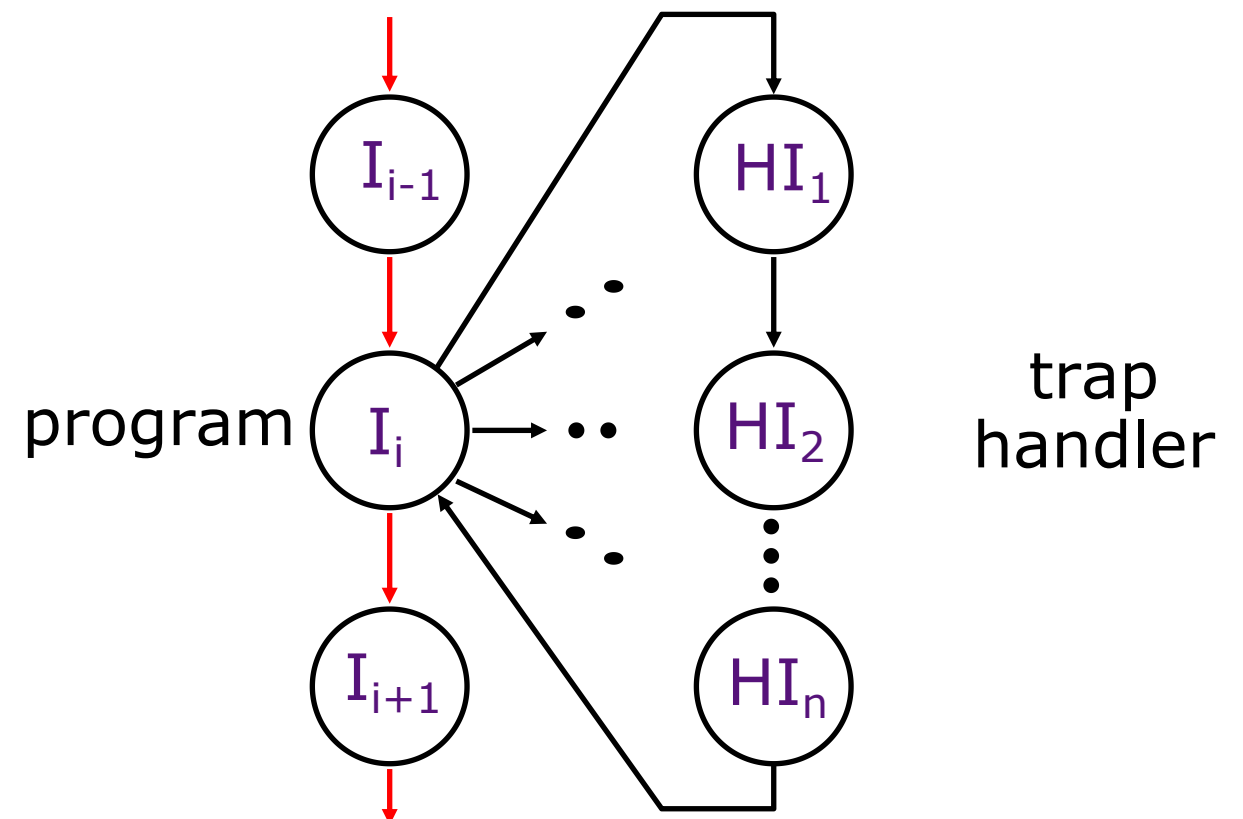
In CA (you'll see other definitions in use elsewhere):

- Interrupt – caused by an event *external* to current running program (e.g. key press, mouse activity)
 - *Asynchronous* to current program, can handle interrupt on any convenient instruction
- Exception – caused by some event during execution of one instruction of current running program (e.g., page fault, bus error, illegal instruction)
 - Synchronous, must handle exception on instruction that causes exception
- Trap – action of servicing interrupt or exception by hardware jump to “trap handler” code

Traps, Interrupts & Exceptions

- Altering the normal flow of control

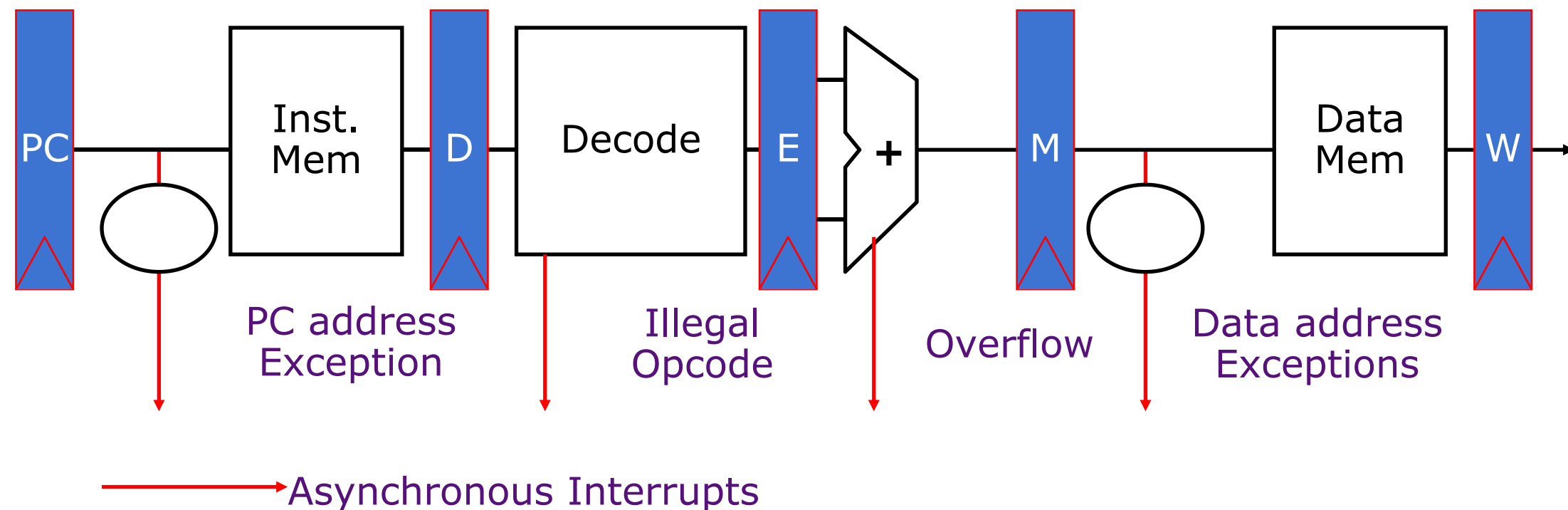
An external or internal event that needs to be processed-by another program-the OS. The event is often unexpected from original program's point of view.



Precise Traps

- Trap handler's view of machine state is that every instruction prior to the trapped one has completed, and no instruction after the trap has executed.
- Implies that handler can return from an interrupt by restoring user registers and jumping back to interrupted instruction (Supervisor exception program counter (**SEPC**) register will hold the instruction address)
 - Interrupt handler software doesn't need to understand the pipeline of the machine, or what program was doing!
 - More complex to handle trap caused by an exception than interrupt
- Providing precise traps is tricky in a pipelined superscalar out-of-order processor!
 - But handling imprecise interrupts in software is even worse.

Trap Handling in 5-stage Pipeline

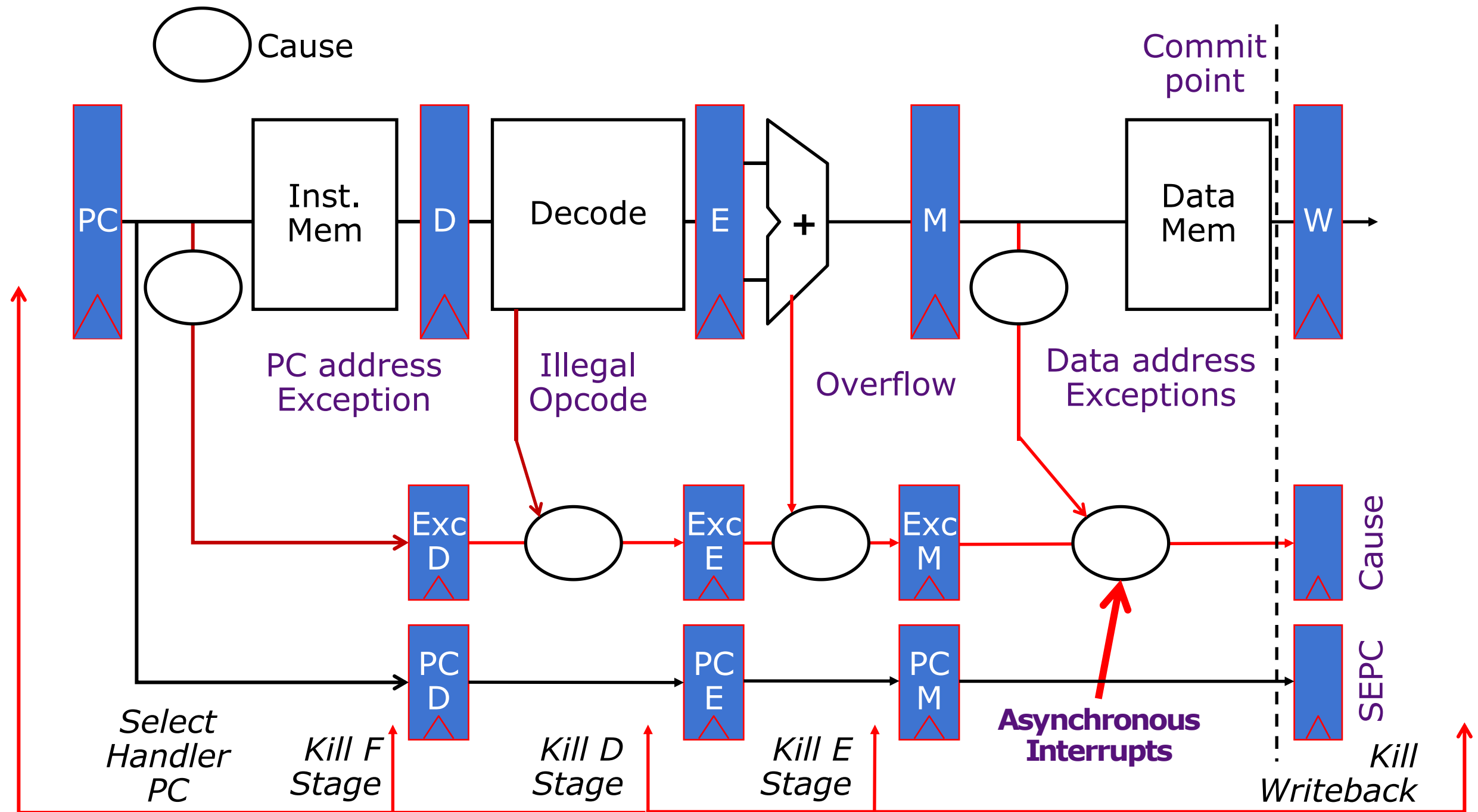


- How to handle multiple simultaneous exceptions in different pipeline stages?
- How and where to handle external asynchronous interrupts?

Handling Traps in In-Order Pipeline

- Hold exception flags in pipeline until **commit point** (M stage)
- Exceptions in earlier instructions override exceptions in later instructions
- Exceptions in earlier pipe stages override later exceptions *for one given instruction*
- Inject external interrupts at commit point
- If exception/interrupt at commit: update Cause and SEPC registers, kill all stages, inject handler PC into fetch stage

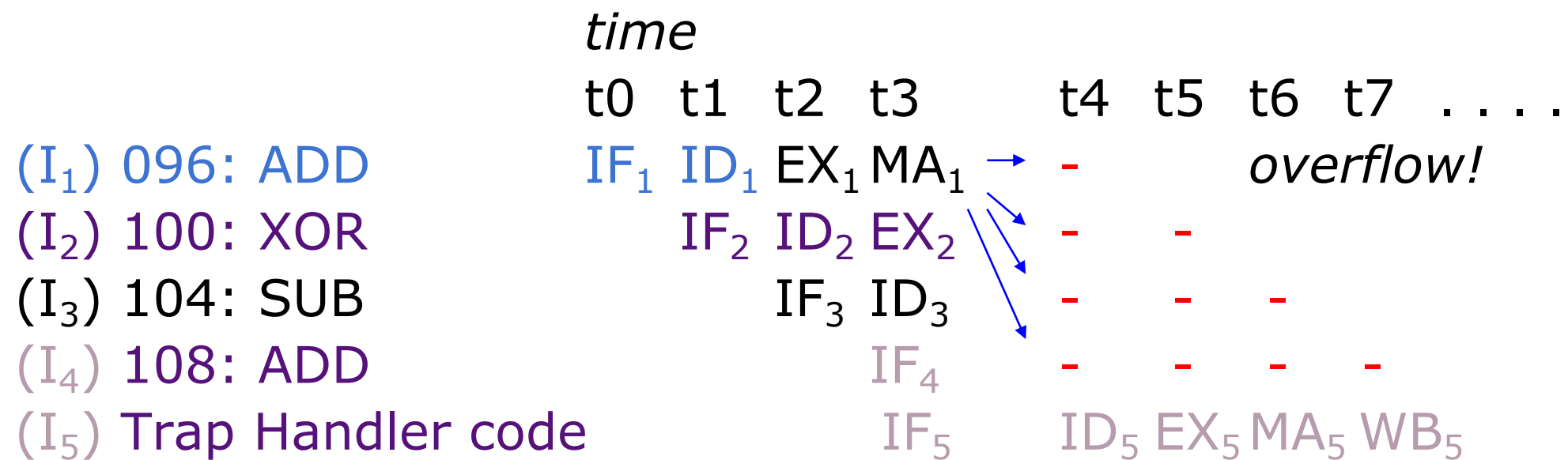
Save Exceptions until Commit



Handling Traps in In-Order Pipeline

- Hold exception flags in pipeline until **commit point** (M stage)
- Exceptions in earlier instructions override exceptions in later instructions
- Exceptions in earlier pipe stages override later exceptions *for a given instruction*
- Inject external interrupts at commit point
- If exception/interrupt at commit: update Cause and SEPC registers, kill all stages, inject handler PC into fetch stage

Trap Pipeline Diagram



What does the OS do?

- One of the first things that runs when your computer starts (right after firmware/bootloader)
- Loads, runs and manages programs
- I/O with the rest of computer

Launching Applications

- Applications are called “processes” in most OSes.
 - Process: separate memory;
 - Thread: shared memory
- Created by another process calling into an OS routine (using a “**syscall**”, more details later).
 - Depends on OS, but Linux uses **fork** to create a new process, and **execve** to load application.
- Loads executable file from disk (using the file system service) and puts instructions & data into memory (.text, .data sections), prepare stack and heap.
- Set argc and argv, jump into the main function.

Processes Management

- The OS manages **multiprogramming**.
 - **Multiplexing** (i.e., running “simultaneously”) multiple applications (processes) on one CPU.
 - This is achieved via process-level **context switches**, i.e., switches between processes very quickly (on the human time scale)
- The OS also manages **multiprocessing**.
 - Running processes simultaneously on different CPUs.
- The OS also manages processes calling other programs.
 - Current process requests this from the OS via a **syscall**.

System Call: Request OS Service (1/2)

- A **system call (syscall)** is a “**software interrupt**” that allows a **(user) process** to request a service from the operating system.
 - Similar to a function call, except now executed by the kernel.
- Example service requests:
 - Creating and deleting files; reading/writing files;
 - Accessing external devices (e.g., scanner);
 - `printf`, `malloc`, etc. (`ecalls` in RISC-V); etc.
 - Launch a new process

System Call: Request OS Service (2/2)

- A **system call (syscall)** is a “**software interrupt**” that allows a **(user) process** to request a service from the operating system.
 - Similar to a function call, except now executed by the kernel.
- Example service requests:
 - Creating and deleting files; reading/writing files;
 - Accessing external devices (e.g., scanner);
 - printf, malloc, etc. (ecalls in RISC-V); etc.
 - Launch a new process
- Suppose a user process (e.g., Linux shell) wants to launch a new app:
 - Shell **forks**: a syscall that traps into the OS kernel process
 - OS (**supervisor mode**): Loads program (**see CALL**); jumps to start of new app's main. Returns to user mode.
 - Shell: “waits” for new app's main to return (**joins**)

Multiprogramming

- The OS runs multiple applications at the same time.
- But not really (unless you have a core per process)
 - Time-sharing processor
- When jumping into process, set timer interrupt.
 - When it expires, store PC, registers, etc. (process state).
 - Pick a different process to run and load its state.
 - Set timer, change to user mode, jump to the new PC.
- Switches between processes very quickly. This is called a “context switch”.
- Deciding what process to run is called **scheduling**.

Protection, Translation, Paging

- Supervisor mode does not fully isolate applications from each other or from the OS.
 - Application could overwrite another application's memory.
 - Also, may want to address more memory than we actually have (e.g., for sparse data structures).
- Solution: **Virtual Memory** (Next lecture). Gives each process the illusion of a full memory address space that it has completely for itself.

Summary

- Once we have a basic machine, it's mostly up to the OS to use it and define application interfaces.
- Main functions of an OS
 - Startup the computer (boot)
 - I/O by polling or interrupt
 - Exception, interrupt, trap
 - Precise trap
 - Resources management, etc.
 - More to explore in CS 130